

1. Introduction

The increasing availability of digital research data repositories has created new opportunities for large-scale data collection and secondary analysis. However, identifying and retrieving relevant qualitative research datasets from such repositories remains a challenging task due to large data volumes and heterogeneous metadata structures.

This work focuses on the automated identification and extraction of datasets related to qualitative data analysis (QDA). A web-based data retrieval approach was developed to systematically search for QDA-related projects, download complete datasets, and store both files and metadata in a structured format for further analysis.

Two research data repositories were explored in this study:

The Murray Research Archive (<https://www.murray.harvard.edu/>) is a specialized archive for social science research data, particularly longitudinal and qualitative studies. It operates as a domain-specific repository within the broader Harvard data infrastructure.

The International Household Survey Network (IHSN) (<https://ihsn.org/>) is a global data platform that provides access to survey and census microdata, primarily focusing on socio-economic and demographic research.

The Murray Research Archive (Repository #18) is part of the larger Harvard Dataverse (Repository #10). Both repositories use the same Dataverse platform and share the same technical infrastructure and website framework. As a result, projects extracted from these two repositories may largely overlap, since Murray is effectively a sub-repository within Harvard Dataverse.

By exploring both repositories, it was possible to evaluate the consistency of the extraction method across similar platforms while also addressing repository-specific challenges.

2. Methods

Search Strategy

A list of file extensions related to qualitative data analysis (QDA) software was first created. These extensions represent project files generated by different QDA tools. The list was stored in a CSV file and used as the main input for the search process.

In the first step, a query was constructed using all collected file extensions. The search was performed through the repository API by combining extensions. In this way, all files matching at least one of the QDA extensions could be identified.

Once a matching file was found, the corresponding dataset (project) was not partially downloaded. Instead, the complete dataset was retrieved. This approach ensures that all related files and metadata belonging to a project are preserved.

QDA File Extensions

The following table shows the QDA software tools and their associated file extensions used in the search process.

Software	File Extensions
REFI-QDA	.qdpX
MaxQDA	.MQDA, .MQTC, .MX24, .MC24, .MX22, .MX20, .MX18, .MX12, .MX11, .MX5, .MX4, .MX3, .MX2, .M2K
NVivo	.nvp, .nvpX, .nvivo
ATLAS.ti	.atlproj, .atlprojbundle, .atl, .hpr7
QDA Miner	.ppj
audiotranskription	.qdc, .f4project
Quirkos	.qrk

Exact-Match Keyword-Based Search

In addition to extension-based search, a second search phase was implemented using keyword queries. A list of predefined phrases used. These phrases were combined into an exact-match query and used to search for datasets directly. An exact-match query was used to avoid results that contain only a part of search keyword. For example, a normal search for the keyword “qualitative coding” will return all the dataset which mention qualitative or coding somewhere. This will return mostly irrelevant data which is not desirable.

A list of keywords was created to be used as search term.

qualitative coding	qualitative codes	qualitative code	qualitative coded
Dedoose	REFI-QDA	Maxqda	ATLAS.ti
QDA	audiotranskription	Quirkos	coded transcripts
thematic analysis	Qualitative Data	CAQDAS	qualitative research
coded interviews	coded transcripts	interveiw Data	interveiw transcripts

File Storage Structure

All downloaded datasets are stored in a structured directory hierarchy. Each dataset is assigned a unique folder, which contains all successfully downloaded files belonging to that dataset.

The general folder structure is illustrated below:

```
Downloaded_Files/
|
|— harvard-murray-archive/
|   |— file1.ext
|   |— file2.ext
|   |— ...
|
|— IHSN/
|   |— fileA.ext
|   |— fileB.ext
|   |— ...
```

Each dataset folder is named using its repository-specific identifier. This makes it possible to link the local copy directly to the original dataset.

Metadata Storage

In addition to file storage, metadata is stored in a relational SQLite database. The database is used to document dataset-level and file-level information in a structured and queryable form. This allows systematic tracking of download status, authorship, licensing, and dataset details.

The database consists of four main tables: PROJECTS, FILES, PERSON_ROLE, and LICENSES.

PROJECTS Table

This table stores general information about each dataset.

Column Name	Description
id	Unique internal identifier of the dataset
query_string	Search query that led to dataset discovery
repository_id	Identifier of the source repository
repository_url	URL of the repository
project_url	Direct link to the dataset
version	Dataset version number
title	Title of the dataset
description	Dataset description
language	Language information (if available)
doi	Persistent DOI or identifier
upload_date	Original upload date
download_date	Date of local download
download_repository_folder	Name of repository folder
download_project_folder	Local dataset folder name
download_method	Method used for download

FILES Table

This table stores information about each file within a dataset.

Column Name	Description
id	Unique internal identifier of the file

project_id	Reference to associated dataset
file_name	Name of the file
file_type	File extension
status	Download status (SUCCEEDED, FAILED, SKIPPED). Every file still gets a FILES row even if it can't be downloaded; in that case status is FAILED
note	In case of FAILED note is set to: RESTRICTED when the API marks the file restricted. Video or Audio File, which usually have very large size.

PERSON_ROLE Table

This table stores information about dataset authors and their roles.

Column Name	Description
id	Unique internal identifier
project_id	Reference to associated dataset
name	Name of the person
role	Role in the dataset (e.g., Author)

LICENSES Table

This table stores licensing information for each dataset.

Column Name	Description
id	Unique internal identifier
project_id	Reference to associated dataset
license	Name of dataset license

3. Discussion and Issues

During the implementation and data collection process, several practical issues were encountered. Each issue required a specific solution in order to ensure stable and consistent dataset retrieval.

Folder Naming Problem

It was observed that dataset names (project names) could not reliably be used as folder names. Some dataset names contained characters that are not allowed in file system paths. In addition, certain names were excessively long, which could cause technical problems depending on the operating system.

To solve this issue, the unique dataset identifier (project ID) was used as the folder name instead of the dataset title. Since the identifier is guaranteed to be unique and system-compatible, this approach ensures consistent and error-free folder creation.

External Repository Redirection

During the data collection process, it was observed that some datasets were not hosted directly on Harvard Dataverse but were stored in external Dataverse-based repositories. As a result, metadata retrieval failed when only the primary Harvard Dataverse API endpoint was used.

The following external repositories were encountered:

- Borealis Data (borealisdata.ca)
- DataverseNL (<https://dataverse.nl>)
- Data Stations Physical and Technical Sciences (<https://phys-techsciences.datastations.nl>)

To address this, the code was adapted to include a fallback API call. If the metadata could not be retrieved from the primary repository, a second request was sent to the external repository. After successful retrieval of metadata from the external source, file downloads were performed using the corresponding external URLs.

This solution ensured that datasets hosted outside the main repository were not excluded from the collection.

Search Query Precision

Initial search queries did not always return the expected results. For example, the query *qualitative coding* returned datasets that contained the word *code* but were unrelated to qualitative research.

A similar issue was observed with the query *qualitative research*. When searched in the normal way, the API returned 161,126 datasets. However, from around dataset 2000 onward, the results started to become clearly irrelevant. Many datasets were included only because their descriptions contained common words such as *data* or *quality*, without being related to qualitative research.

To address this problem, the API search was modified to enforce exact phrase matching. By using quotation marks around the search phrase, only datasets containing the full phrase were returned. In the case of *qualitative research*, enforcing the exact phrase significantly reduced the total number of returned datasets. Although fewer datasets were retrieved, the majority of them were directly relevant to qualitative research.

This adjustment improved precision and reduced noise in the dataset. As a result, irrelevant projects were filtered out early, and the final collection remained more focused on the intended research domain.

Missing Descriptions in IHSN

In the IHSN repository, many projects did not contain a formal description field. However, some projects included an abstract.

When a description was missing, the available abstract was used instead as the project description. This ensured that each dataset entry in the database contains at least minimal contextual information.

Keyword Handling

Many projects contained multiple keywords separated by commas. If stored as a single string, later keyword-based searches would become inefficient.

To improve searchability, each keyword was stored in a separate row. This normalization allows individual keywords to be queried independently and simplifies later analysis.

File-Level Download Status Tracking

Each file listed in the dataset metadata receives a corresponding row in the FILES table, regardless of whether the file was successfully downloaded.

Three status values are recorded:

- **SUCCEEDED** – File downloaded successfully
- **FAILED** – File could not be downloaded
- **SKIPPED** – File intentionally not downloaded

If a file cannot be downloaded, the status is marked as SKIPPED and additional information is recorded. The following cases are distinguished:

- **RESTRICTED** – The API indicates that the file is restricted
- **Video or Audio File** – Large multimedia files are intentionally excluded
- **Too large file size: <X.XX> GB** – if the file is larger than 500MB size cap. Usually image datasets fall in this category.

This design ensures transparency. Even files that are not downloaded remain documented, allowing later review and analysis of download limitations.

Results

The data retrieval process was conducted on two repositories. The objective was to collect datasets related to qualitative research while minimizing the inclusion of irrelevant projects. Through the use of file-extension filtering and exact-match keyword queries, it was attempted to include mostly relevant datasets.

The following table summarizes the number of retrieved projects and files for each repository:

Repository	Number of Projects	Number of Files
Harvard Repository	602	9,559
IHSN Repository	1,447	6,998
Total	2,049	16,557

In total, **2,049 projects** were retrieved across both repositories, containing **16,557 files**.

Although the IHSN repository yielded a higher number of projects, the Harvard repository contained a larger number of files overall. The applied filtering strategy helped to focus the retrieval process on datasets that are more likely to be relevant to qualitative research.